University of Science

Faculty of Information Technology

Data Structures and Algorithms Project Report

Topic: STRING MATCHING

Course: Data Structures & Algorithms

Instructor:
Le Thanh Tung
Tran Hoang Quan

Class: 25C10
Group ID: 07

Group Members:
Dang Ba Lam - 25127403
Hua Minh Khang - 25127073
Trinh Bao Khoi - 25127256
Nguyen Huu Ngoc Thuan - 25127153

Ho Chi Minh City, March 13, 2026

Contents

1 Group Information **2**

2 Problem Statement **3**

3 Algorithms Analysis **4**
3.1 The Naïve string-matching algorithm 4
3.2 The Boyer Moore Algorithm 8
3.3 step trace 9
3.4 The Rabin-Karp Algorithm 13

4 Experimental Results **16**
4.1 Running time 16
4.2 The number of comparisons 16
4.3 Tables and Graphs 16

5 Acknowledgements **29**

6 References **30**

1 Group Information

Student Name	**ID**	**Assigned Tasks**	**% Completed**
Dang Ba Lam	25127403	Leader, Command line arguments, Implementation & Report (The Naive approach, Experimental results)	25%
Hua Minh Khang	25127073	Setting Algorithm mode and Comparisons mode, Make input for Input Size, Implementation & Report (The Knuth-Morris-Pratt algorithm, The Aho Corasick algorithm Acknowledgement)	25%
Trinh Bao Khoi	25127256	Data Generator, Implementation & Report (The Boyer-Moore algorithm, Project organization)	25%
Nguyen Huu Ngoc Thuan	25127153	Adjust Comparisons and Time, Implementation & Report (The Rabin-Karp algorithm, Title Page, Table of Contents, Team Members List)	25%
		Total Group Completion	**110%**

Table 1: Group Information

**Note: All members participated in Time Calculator implementation, Report writing, Citations, and Final checking.*

2 Problem Statement

STRING matching is a problem that find all occurrences of a pattern in the text. This problem is an indispensable technique in computer science for solving various crucial problems, such as text editing, text searching, and symbol manipulation.[3]

The string matching problem is defined as follows: We assume that the text is a string $T[1..n]$ of length n and the pattern is a string $P[1..m]$ of length m which $m \leq n$. The problem is to find all occurrences of P in T by shifting P along T such that $0 \leq s \leq n - m$ (s is the shift amount) and $T[s+1..s+m] = P[1..m]$ (that is, if $T[s+i] = P[i]$, for $1 \leq i \leq m$).[2]

For instance:

- If $T =$ "ababcab" and $P =$ "abc", then the output is $(3 \rightarrow 5)$ because the pattern occurs at position 3 in the text.

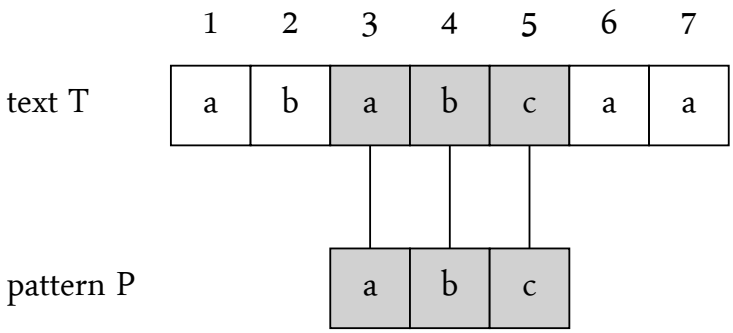


Figure 1: T = "ababcab" and P = "abc"

- If $T =$ "aaaaa" and $P =$ "aa", then the output is $(1 \rightarrow 2)$, $(2 \rightarrow 3)$, $(3 \rightarrow 4)$, and $(4 \rightarrow 5)$ because the pattern occurs at positions 1, 2, 3, and 4 in the text.

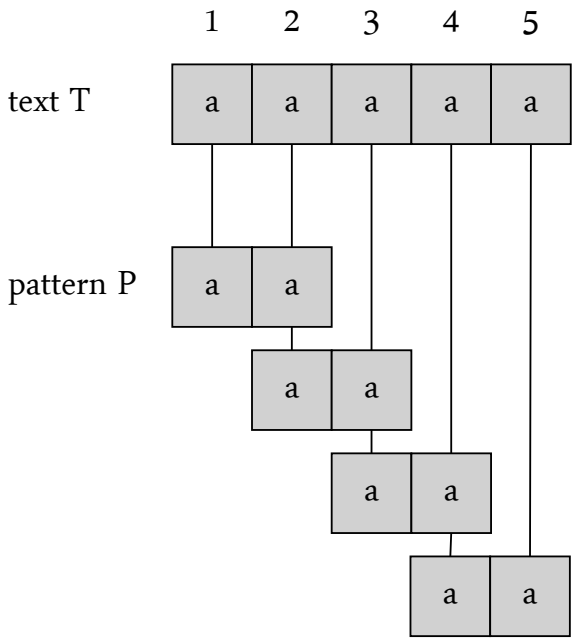


Figure 2: T = "aaaaa" and P = "aa"

3 Algorithms Analysis

3.1 The Naïve string-matching algorithm (Brute-Force approach)

3.1.1 Introduction & Application

- **Core idea:** According to Abdeen[1], the brute-force algorithm works with two pointers; a "text pointer" i and a "pattern pointer" j. For all possible shifts of the pattern P over the text T, compare the pattern characters with the corresponding text characters. If mismatch occurs, i is incremented by 1, and j is reset to 0 and the comparing process is restarted. In the case that all characters match, the algorithm records the position of the match and continues searching for the next possible match.

- **Real-world applications:**
  - Text Editors: Using in "Search" or "Find and Replace" features to locate destination words.
  - Bioinformatics: Searching for specific DNA.
  - Checking for Plagiarism: Comparing documents to find and compare a script or a piece of code with the exact match in a large database.

3.1.2 Pseudocode

Notation:

- **Text T (R rows, C columns)**: The main text being searched.
- **Pattern P (K the number of keywords)**: The list of strings that we are looking for.
- **Positions ([(start position, end position), ...])**: A list storing the indices where a complete match is found.

```
BRUTE-FORCE-ALGORITHM(T, P, K, R, C)
pos = empty array of size K
for idx = 1 to K do
    keyword = P[idx]
    m = keyword.length

    ▷ Search Horizontally
    for i = 1 to R do
        for j = 0 to C − m do
            if keyword[1 ... m] == T[i, j + 1 ... j + m] then
                add position to pos[idx]
            end if
        end for
    end for

    ▷ Search Vertically
    for i = 1 to C do
        for j = 0 to R − m do
            if keyword[1 ... m] == T[j + 1 ... j + m, i] then
                add position to pos[idx]
```

```
            end if
        end for
    end for
end for
return pos
```

3.1.3 Step-by-step trace[2]

Notation:

- **Text** T **(length** n**)**: The main text being searched.
- **Pattern** P: The string that we are looking for.
- **Shift** (s): The current offset of the text T.
- **Positions ([(start position, end position), . . .])**: A list storing the indices where a complete match is found.

Example: text $T = [a, a, b, c, a, b, b]$ and pattern $P = [a, b, c]$

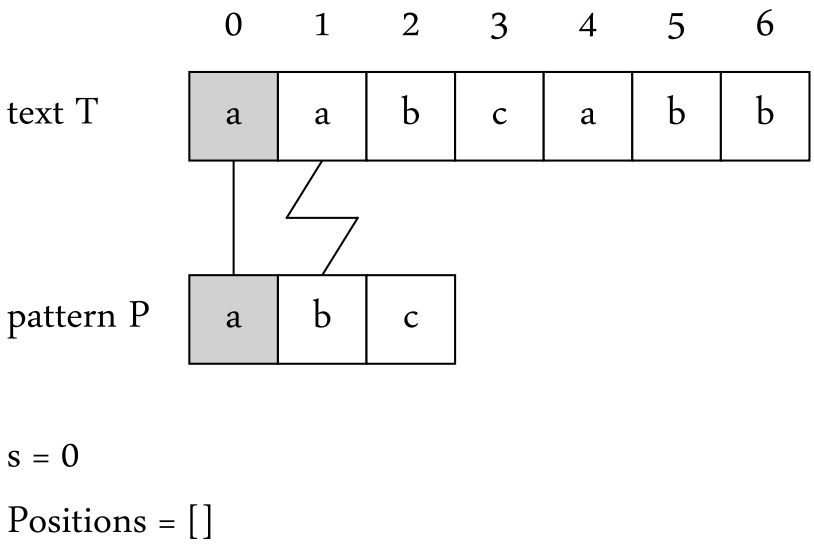


Figure 3: Step 1

- The first character 'a' matches in both arrays.

  The second character in text T ('a') does not match the second character in pattern P ('b').

  Text at the bottom shows the current shift is $s = 0$ and the list of found matches is empty ($Positions = []$).

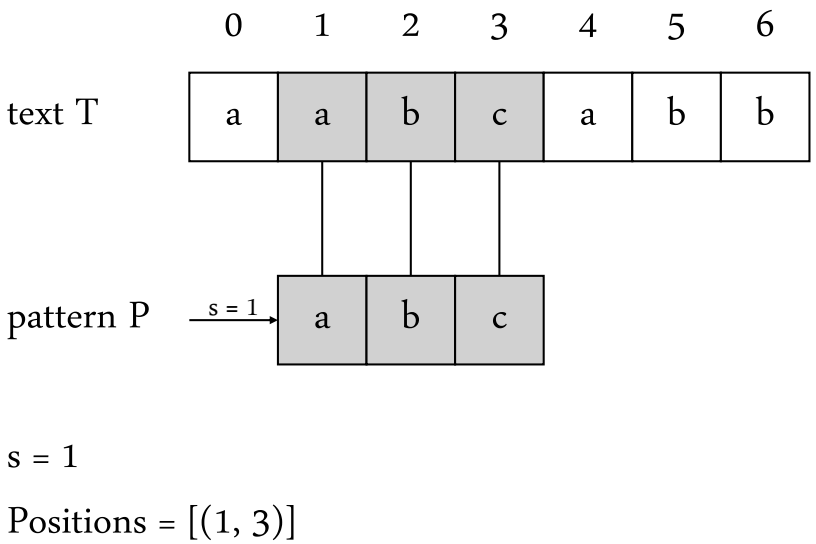


Figure 4: Step 2

- All characters in pattern P ('a', 'b', 'c') successfully match the corresponding characters in text T ('a', 'b', 'c').

  Shift is $s = 1$ and the list of found matches is updated to record this position ($Positions = [(1, 3)]$).

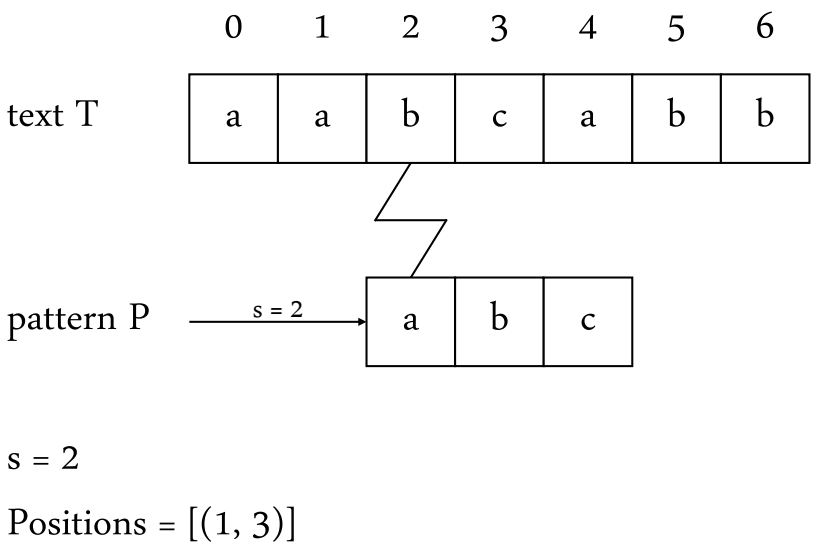
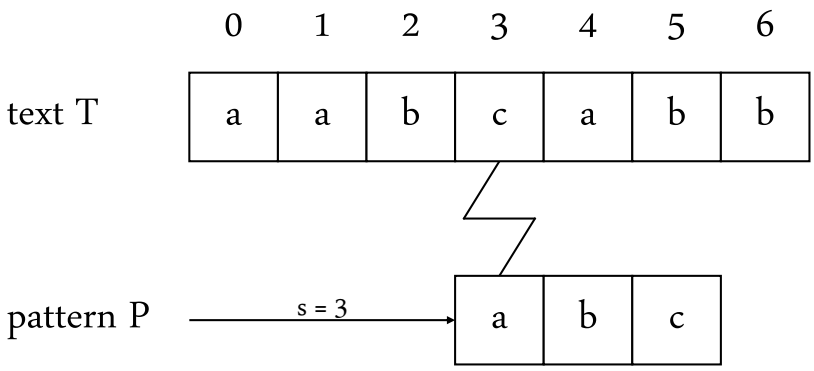


Figure 5: Step 3

- The first character in pattern P ('a') does not match the character in text T ('b') at the current shift position $s = 2$.

  Skip to the next shift without updating $Positions$.



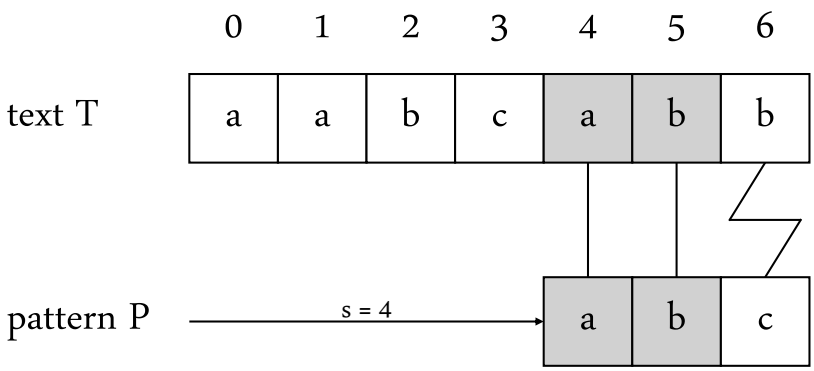
s = 3

Positions = [(1, 3)]

Figure 6: Step 4

- The first character in pattern P ('a') does not match the character in text T ('c') at the current shift position $s = 2$.

  Skip to the next shift without updating *Positions*.



s = 4

Positions = [(1, 3)]

Figure 7: Step 5

- The first two characters in pattern P ('a', 'b') match the corresponding characters in text T, but the third character in pattern P ('c') does not match the character in text T ('b') at the current shift position $s = 4$.

  Skip to the next shift without updating *Positions*.

- After that, s increases by 1. Because $s = 5$ and $n - m = 7 - 3 = 4$, we have $s > n - m$. The algorithm terminates and returns the list of found matches: Positions $= [(1, 3)]$.

3.1.4 Complexity

Metric	Complexity	Reasoning
Time (Worst)	$O(K \cdot m \cdot R \cdot C)$	Occurs when patterns and text share many prefix characters (e.g., all 'A's).
Time (Average)	$O(K \cdot R \cdot C)$	In practice, a mismatch usually occurs after only a few character comparisons.
Time (Best)	$O(K \cdot R \cdot C)$	The first character of the pattern always results in a mismatch at every shift.
Auxiliary Space	$O(1)$	The algorithm uses only a constant amount of extra memory for loop indices.

Table 2: Complexity Analysis of 2D Brute Force Algorithm

3.1.5 Discussion

- In scenarios 1, the number of patterns K is small, and the text is from 10×10 to 500×500. Brute-force approach is efficient and likely linear running time $O(R \cdot C)$.
- In scenarios 2, the number of patterns K is large, and the text is fixed at 100×100. This scenario is more challenging for the brute-force approach, as the running time multiplies by K times, which is much longer compared to scenario 1.
- In conclusion, the brute-force approach is more suitable for scenarios with a small number of patterns. It is worse for scenarios with a large number of one.

3.2 The Boyer Moore

3.2.1 Introduction & Application

- **Core idea:** The Boyer Moore Algorithm is a practical method to find patterns. This algorithm is the combination of three main ideas comprising scanning the pattern in revserse order, bad character heuristic and good suffix heuristic. This speeds up the searching process considerably by skipping a plenty of characters.
- **Reality applications:**
  - Text Editors: Using in "Search" or "Find and Replace" features to locate the destination of the words.
  - Intrusion Detection Systems (Network Security): use Boyer-Moore to scan network traffic in real-time for "malicious signatures"—specific byte patterns that indicate a cyberattack or virus.
  - Large-Scale Plagiarism Detection: compare submitted essays against billions of web pages and academic journals to find copied content.

3.2.2 Pseudocode

Notation:

- **Text T (R rows, C columns)**: The 2D character matrix.
- **Pattern P (K keywords)**: The list of strings to search.
- **Heuristics**: BC (Bad Character table) and GS (Good Suffix table).

```
BOYER-MOORE ALGORITHM(T, P, K, R, C)
pos = empty array of size K
for idx = 1 to K do
    P_i = P[idx], m = P_i.length
    BC = PreProcessBadChar(P_i)
    GS = PreProcessGoodSuffix(P_i)
  ▷ Horizontal Search (Rows)
    for r = 0 to R − 1 do
        s = 0
        while s ≤ C − m do
            j = m − 1
            while j ≥ 0 and P_i[j] == T[r, s + j] do
                j = j − 1
            end while
            if j < 0 then
                add (r, s) to pos[idx]
                s = s + GS[0]
            else
                s = s + max(GS[j], j − BC[T[r, s + j]])
            end if
        end while
    end for
  ▷ Vertical Search (Columns)
    for c = 0 to C − 1 do
        s = 0
        while s ≤ R − m do
            j = m − 1
            while j ≥ 0 and P_i[j] == T[s + j, c] do
                j = j − 1
            end while
            if j < 0 then
                add (s, c) to pos[idx]
                s = s + GS[0]
            else
                s = s + max(GS[j], j − BC[T[s + j, c]])
            end if
        end while
    end for
end for
return pos
```

3.3 step trace

Notation:

- **Text T (length n)**: The main text being searched.
- **Pattern P**: The string that we are looking for.
- **Shift (s)**: The current offset of the text T.
- **Positions ([(start position, end position), ...])**: A list storing the indices where a complete match is found.

Example: text $T = [G, C, A, T, G, C, C, T, A, G]$ and pattern $P = [T, A, T, G]$

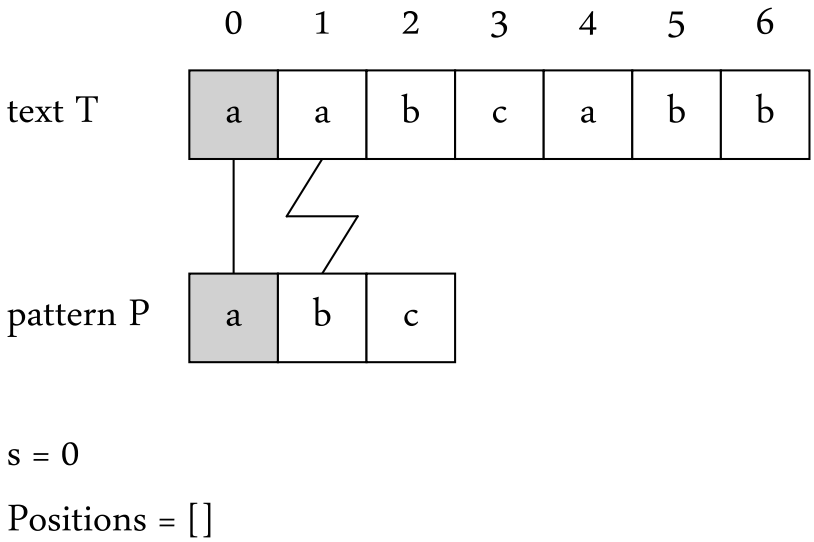
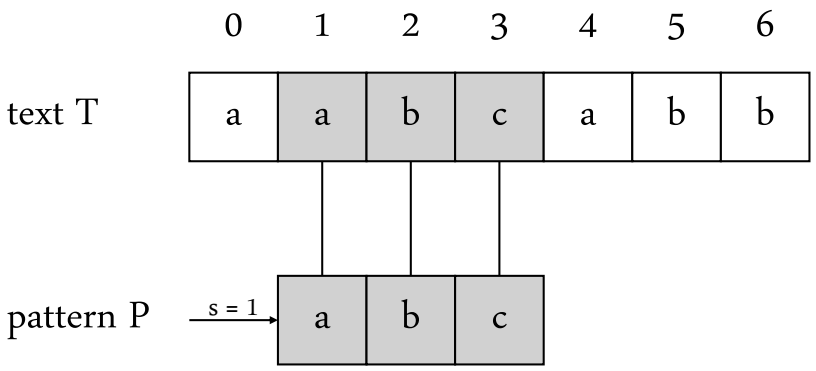


Figure 8: Step 1

- The first character 'a' matches in both arrays.

  The second character in text T ('a') does not match the second character in pattern P ('b').

  Text at the bottom shows the current shift is $s = 0$ and the list of found matches is empty ($Positions = []$).



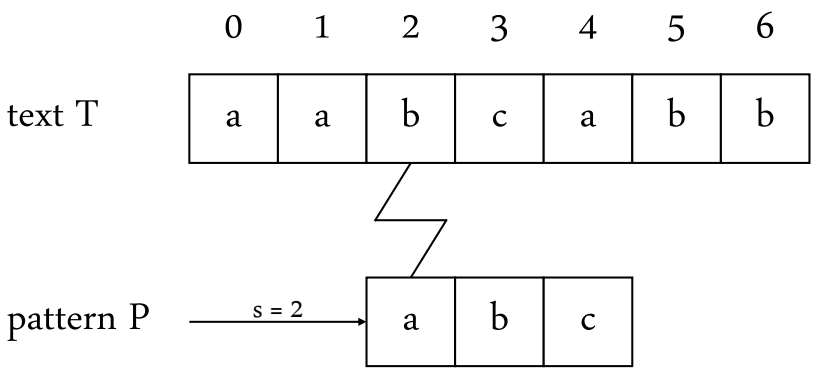
s = 1

Positions = [(1, 3)]

Figure 9: Step 2

- All characters in pattern P ('a', 'b', 'c') successfully match the corresponding characters in text T ('a', 'b', 'c').

  Shift is $s = 1$ and the list of found matches is updated to record this position ($Positions = [(1, 3)]$).



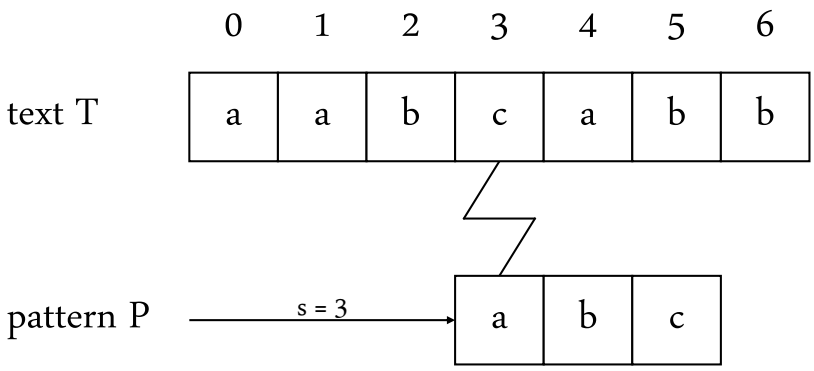
s = 2

Positions = [(1, 3)]

Figure 10: Step 3

- The first character in pattern P ('a') does not match the character in text T ('b') at the current shift position $s = 2$.

  Skip to the next shift without updating $Positions$.



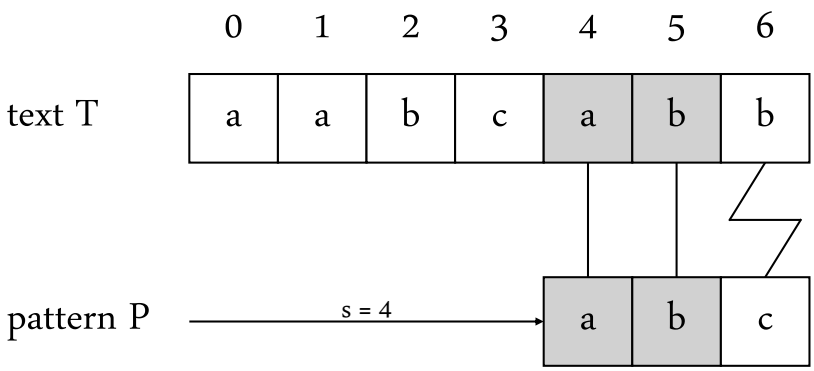
s = 3

Positions = [(1, 3)]

Figure 11: Step 4

- The first character in pattern P ('a') does not match the character in text T ('c') at the current shift position $s = 2$.

  Skip to the next shift without updating *Positions*.



s = 4

Positions = [(1, 3)]

Figure 12: Step 5

- The first two characters in pattern P ('a', 'b') match the corresponding characters in text T, but the third character in pattern P ('c') does not match the character in text T ('b') at the current shift position $s = 4$.

  Skip to the next shift without updating *Positions*.

- After that, s increases by 1. Because $s = 5$ and $n - m = 7 - 3 = 4$, we have $s > n - m$. The algorithm terminates and returns the list of found matches: Positions $= [(1, 3)]$.

3.3.1 Complexity

Metric	Complexity	Reasoning
Time (Worst)	$O(K \cdot m \cdot R \cdot C)$	Occurs when patterns and text share many prefix characters (e.g., all 'A's).
Time (Average)	$O(K \cdot R \cdot C)$	In practice, a mismatch usually occurs after only a few character comparisons.
Time (Best)	$O(K \cdot R \cdot C)$	The first character of the pattern always results in a mismatch at every shift.
Auxiliary Space	$O(1)$	The algorithm uses only a constant amount of extra memory for loop indices.

Table 3: Complexity Analysis of 2D Brute Force Algorithm

3.3.2 Discussion

- In scenarios 1, the number of patterns K is small, and the text is from 10×10 to 500×500. Brute-force approach is efficient and likely linear running time $O(R \cdot C)$.
- In scenarios 2, the number of patterns K is large, and the text is fixed at 100×100. This scenario is more challenging for the brute-force approach, as the running time multiplies by K times, which is much longer compared to scenario 1.
- In conclusion, the brute-force approach is more suitable for scenarios with a small number of patterns. It is worse for scenarios with a large number of one.

3.4 The Rabin-Karp Algorithm

3.4.1 Introduction & Application

The Rabin-Karp algorithm is a string-matching algorithm that utilizes a hash function to find occurrences of a pattern string P within a text string T. Instead of comparing the pattern character by character against every possible window in the text, it computes a hash value for the pattern and a rolling hash value for each window of the text. If the hash values match, it performs a secondary character-by-character comparison to ensure it is not a hash collision. This rolling hash technique allows the algorithm to update the hash value of the next window in $\mathcal{O}(1)$ time by dropping the first character of the previous window and appending the new character.

Application: Because Rabin-Karp calculates hash values, it is exceptionally efficient when searching for *multiple patterns* simultaneously. In real-world applications, it is widely used in plagiarism detection systems (where multiple sentences or phrases need to be matched against a massive database of documents) and in computational biology for DNA sequence matching.

3.4.2 Pseudocode

The following pseudocode demonstrates the 1D Rabin-Karp matcher. In our C++ implementation for the crossword grid, we utilize $d = 256$ (the number of characters in the extended ASCII set) and a large prime $q = 1000000007$ to minimize hash collisions. We also explicitly handle negative modulo results when updating the hash (as seen in lines 16-18).

Algorithm 1 Rabin-Karp-Matcher(T, P, d, q)

$n \leftarrow$ length$[T]$ $m \leftarrow$ length$[P]$ $h \leftarrow d^{m-1} \bmod q$ $p \leftarrow 0$ Hash value for pattern $t_0 \leftarrow 0$ Hash value for text window $i \leftarrow 1$ **to** m $p \leftarrow (d \cdot p + P[i]) \bmod q$ $t_0 \leftarrow (d \cdot t_0 + T[i]) \bmod q$ $s \leftarrow 0$ **to** $n - m$ $p == t_s$ $P[1..m] == T[s+1..s+m]$ **print** "Pattern occurs with shift " s $s < n - m$ $t_{s+1} \leftarrow (d(t_s - T[s+1] \cdot h) + T[s+m+1]) \bmod q$ $t_{s+1} < 0$ $t_{s+1} \leftarrow t_{s+1} + q$

3.4.3 Step-by-step Trace

We trace the algorithm using the example defined in our Problem Statement: $T =$ "ababcab" and $P =$ "abc". To simplify the mathematical trace, let us map the characters to integers $(a = 1, b = 2, c = 3)$, use a small alphabet size $d = 10$, and a prime number $q = 13$.

- **Pre-computation:**
  - $h = d^{m-1} \bmod q = 10^2 \bmod 13 = 100 \bmod 13 = 9$.
  - $P =$ "abc" $\rightarrow 1, 2, 3$. Hash $p = (1 \cdot 100 + 2 \cdot 10 + 3) \bmod 13 = 123 \bmod 13 = 6$.
- **Window Shifts:**
  - **Shift** $s = 0$**:** $T[1..3] =$ "aba" $\rightarrow 1, 2, 1$.
    $t_0 = 121 \bmod 13 = 4$.
    $t_0 \neq p$ $(4 \neq 6)$. No match.
  - **Shift** $s = 1$**:** $T[2..4] =$ "bab" $\rightarrow 2, 1, 2$.
    $t_1 = (10 \cdot (t_0 - T[1] \cdot h) + T[4]) \bmod q = (10 \cdot (4 - 1 \cdot 9) + 2) \bmod 13 = -48 \bmod 13 \equiv 4$.
    $t_1 \neq p$ $(4 \neq 6)$. No match.
  - **Shift** $s = 2$**:** $T[3..5] =$ "abc" $\rightarrow 1, 2, 3$.
    $t_2 = (10 \cdot (t_1 - T[2] \cdot h) + T[5]) \bmod 13 = (10 \cdot (4 - 2 \cdot 9) + 3) \bmod 13 = -137 \bmod 13 \equiv 6$.
    $t_2 == p$ $(6 == 6)$. We perform a character comparison: "abc" == "abc". **Match found at shift 2!**
  - **Shift** $s = 3$**:** $T[4..6] =$ "bca" $\rightarrow 2, 3, 1$.
    $t_3 = (10 \cdot (t_2 - T[3] \cdot h) + T[6]) \bmod 13 = (10 \cdot (6 - 1 \cdot 9) + 1) \bmod 13 = -29 \bmod 13 \equiv 10$.
    $t_3 \neq p$ $(10 \neq 6)$. No match.
  - **Shift** $s = 4$**:** $T[5..7] =$ "cab" $\rightarrow 3, 1, 2$.
    $t_4 = (10 \cdot (t_3 - T[4] \cdot h) + T[7]) \bmod 13 = (10 \cdot (10 - 2 \cdot 9) + 2) \bmod 13 = -78 \bmod 13 \equiv 0$.
    $t_4 \neq p$ $(0 \neq 6)$. No match.

3.4.4 Complexity

- **Time Complexity:**
  - *Pre-processing:* $\mathcal{O}(m)$ time to compute the hash of the pattern and the first window.
  - *Matching (Best/Average Case):* $\mathcal{O}(n - m + 1)$. The rolling hash computes the next window's hash in $\mathcal{O}(1)$ time. If collisions are rare, the total time is $\mathcal{O}(n + m)$. For our crossword implementation searching a $R \times C$ grid with K keywords, the average time is $\mathcal{O}(R \cdot C \cdot K)$.
  - *Worst Case:* $\mathcal{O}(n \cdot m)$. This occurs when the hash function is poor, resulting in a spurious hit (hash collision) at every single shift, forcing the algorithm to perform m character comparisons each time.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary space, as it only requires a few variables to store the hash values and prime modulo q.

3.4.5 Discussion

When is it good? The Rabin-Karp algorithm shines when we need to search for multiple patterns simultaneously (by pre-calculating the hashes of all patterns). It is highly efficient in practice when a large prime q is chosen, minimizing the chance of hash collisions.

When is it bad? The algorithm degrades gracefully to the Naive Brute-Force approach in the worst-case scenario where hash collisions are frequent. Furthermore, the arithmetic operations involving modulo q can be slower than the simple pointer increments used in algorithms like KMP or Boyer-Moore.

4 Experimental Results

4.1 Running time

The four line graphs below illustrate the running times of 12 sorting algorithms across four input data distributions: **sorted, nearly sorted, reverse sorted, and random**.

Overall, it is evident that **Selection Sort** and **Bubble Sort** exhibit significantly higher execution times compared to the other algorithms. Notably, **Counting Sort** remains the most efficient algorithm across all test cases.

In the sorted and nearly sorted scenarios, **Selection Sort** and **Bubble Sort** consistently record the highest running times.

Regarding the reverse sorted and randomized input, five algorithms show a dramatic increase in execution time: **Insertion Sort**, **Binary Insertion Sort**, **Shaker Sort**, and the two previously mentioned algorithms (i.e., **Selection Sort** and **Bubble Sort**).

4.2 The number of comparisons

The four bar charts below display the number of comparisons made by 12 different categories of sorting algorithms across four input data types: **sorted, nearly sorted, reverse sorted, and random**.

Generally, it is clear that **Selection Sort** and **Bubble Sort** consistently require the highest number of comparisons across all scenarios. In constrast, **Counting Sort** exibits the lowest qunatity of comparison count.

In the sorted and nearly sorted cases, **Selection Sort** and **Bubble Sort** perform the remarkably badly, with their comparion counts being significantly higher than the other algorithms.

In the reverse sorted and randomized input scenarios, the number of comparisons for most $O(n^2)$ algorithms (specifically **Insertion Sort**, **Binary Insertion Sort**, and **Shaker Sort**) increases dramatically. Meanwhile, $O(n \log n)$ algorithms such as **Merge Sort**, **Heap Sort**, and **Quick Sort**, etc. maintain a relatively stable and efficient number of comparisons.

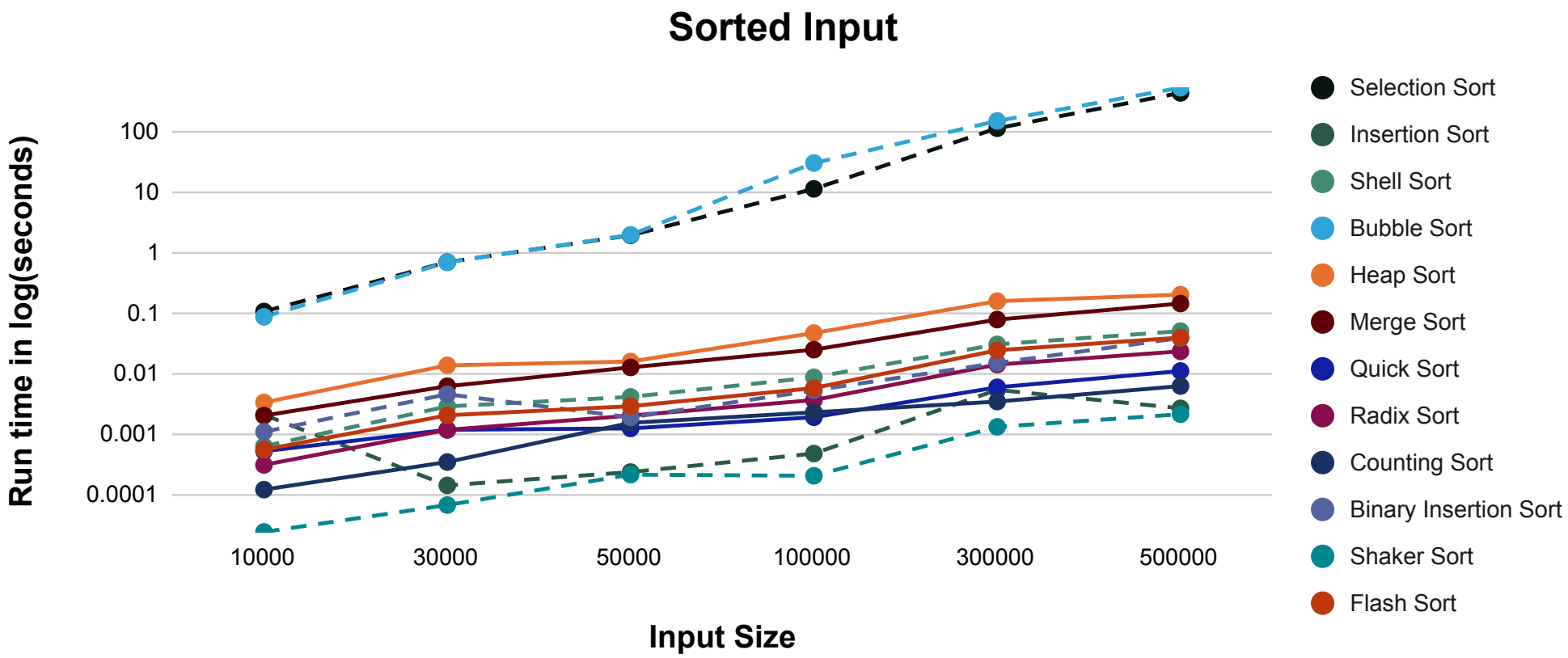
4.3 Tables and Graphs

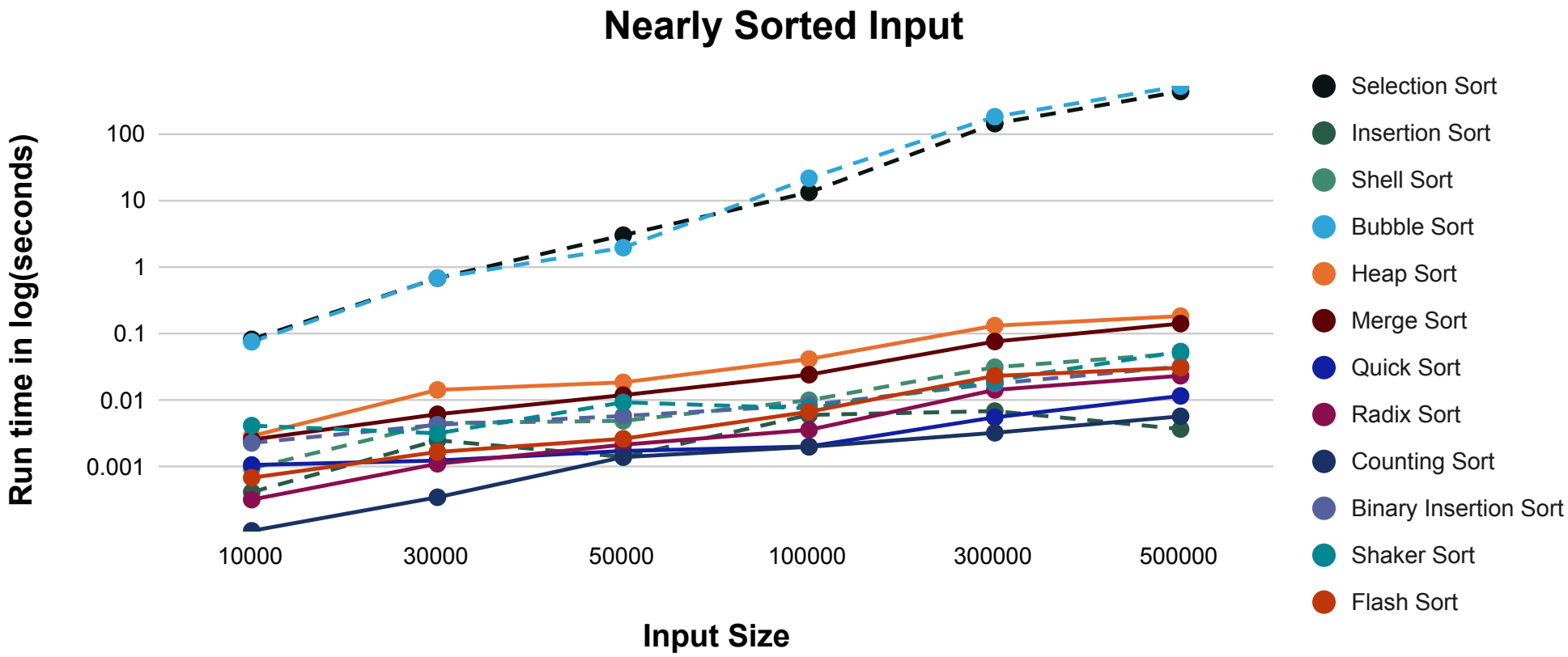
Data order: **Sorted**												
Data size	10.000		30.000		50.000		100.000		300.000		500.000	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision
Selection Sort	105,167	100009999	689,787	900029999	1917,92	2500049999	11233,5	10000099999	112996	90000299999	434892	2,5E+11
Insertion Sort	2,0494	29998	0,1396	89998	0,2322	149998	0,4694	299998	5,3183	899998	2,6309	1499998
Shell Sort	0,6222	360042	2,8272	1170050	4,0681	2100049	8,5708	4500051	30,0859	15300061	49,7864	25500058
Bubble Sort	85,8152	100009999	685,405	900029999	1949,68	2500049999	30057,4	10000099999	149342	90000299999	530168	2,5E+11
Heap Sort	3,289	518705	13,5221	1739633	15,6552	3056481	46,292	6519813	155,882	21431637	199,605	37116275
Merge Sort	1,9953	465243	6,1066	3534740	12,507	11271929	24,5689	25699104	77,2574	62516397	142,383	153959690
Quick Sort	0,5135	244975	1,1606	823646	1,2239	1467264	1,8735	3134498	5,8564	10258249	10,8987	17737894
Radix Sort	0,3049	60107	1,1599	210131	1,9931	350131	3,6059	700131	13,9273	2400155	23,0738	4000155
Counting Sort	0,1188	70005	0,3401	210005	1,513	350005	2,2584	700005	3,4015	2100005	6,1422	3500005
Binary Insertion Sort	1,076	277232	4,501	924464	1,8637	1618928	5,1965	3437856	14,8291	11251424	38,3446	19451424
Shaker Sort	0,0237	20001	0,066	60001	0,2102	100001	0,2014	200001	1,2986	600001	2,0998	1000001
Flash Sort	0,5451	118994	2,0147	356994	2,8529	594994	5,6932	1189994	24,029	3569994	38,9354	5949994

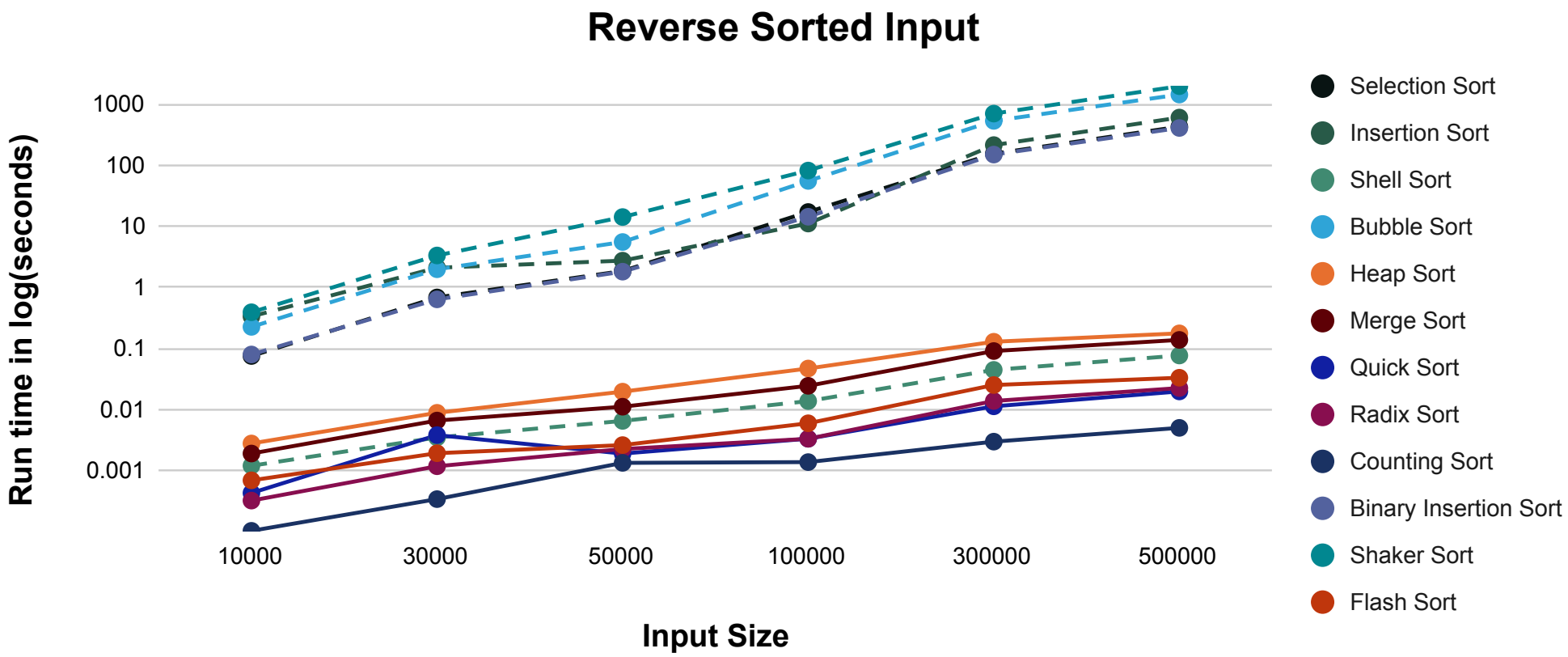
Data order: **Nearly Sorted**												
Data size	10.000		30.000		50.000		100.000		300.000		500.000	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision
Selection Sort	82,4528	100009999	682,385	900029999	3006,82	2500049999	13195,6	10000099999	141754	90000299999	429977	2,5E+11
Insertion Sort	0,4216	174730	2,5244	485886	1,4237	597658	6,0644	877238	6,9249	1442810	3,7205	2032738
Shell Sort	0,9578	360042	4,5803	1170050	4,9382	2100049	10,0758	4500051	31,5097	15300061	50,6101	25500058
Bubble Sort	75,6924	100009999	686,235	900029999	1953,2	2500049999	21510,8	10000099999	180087	90000299999	516635	2,5E+11
Heap Sort	2,9228	518705	14,3961	1739633	18,676	3056481	41,7754	6519813	132,22	21431637	183,995	37116275
Merge Sort	2,5839	964668	6,2076	5147796	12,0211	14036437	24,3352	31436444	76,5169	80950733	142,079	185575804
Quick Sort	1,077	257954	1,2535	855622	1,7467	1507515	2,0375	3184601	5,581	10309309	11,6933	17786083
Radix Sort	0,3253	60107	1,1225	210131	2,1678	350131	3,622	700131	14,3872	2400155	23,4548	4000155
Counting Sort	0,1108	70005	0,3532	210005	1,4122	350005	2,0138	700005	3,2722	2100005	5,7755	3500005
Binary Insertion Sort	2,3063	349914	4,3183	1175428	5,8485	1794400	8,5103	3666698	17,885	11462644	32,2496	19594008
Shaker Sort	4,192	260001	3,2138	780001	9,3725	1700001	7,6127	3400001	19,9086	9000001	54,0995	15000001
Flash Sort	0,694	118970	1,6874	356968	2,6621	594968	6,765	1189970	23,3154	3569970	30,9164	5949971

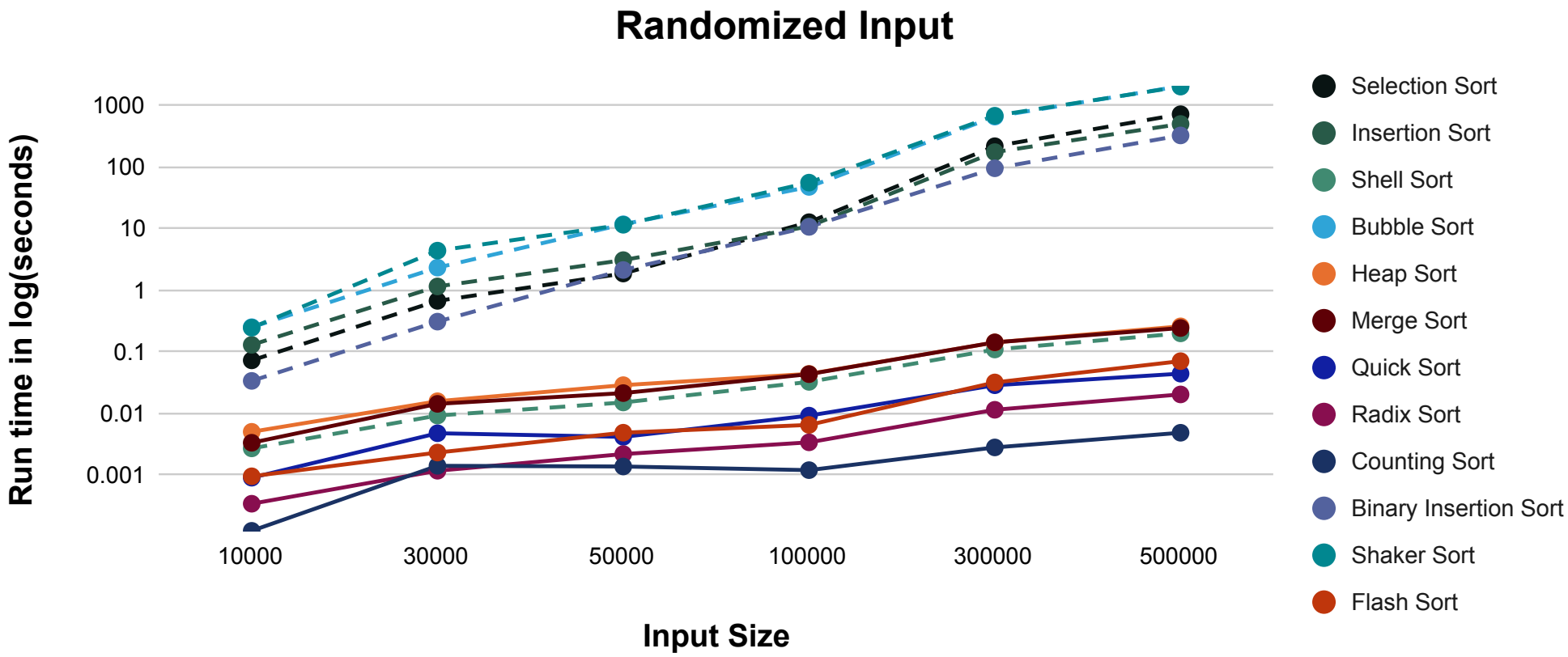
Data order: **Reverse Sorted**												
Data size	10.000		30.000		50.000		100.000		300.000		500.000	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision
Selection Sort	74,7506	100009999	680,337	900029999	1838,97	2500049999	17064,2	10000099999	153636	90000299999	425904	2,5E+11
Insertion Sort	332,46	100009999	2085,1	900029999	2722,32	2500049999	11039,5	10000099999	212837	90000299999	6,01E+05	2,5E+11
Shell Sort	1,1906	360042	3,4599	1170050	6,4618	2100049	13,6358	4500051	44,2755	15300061	76,1319	25500058
Bubble Sort	224,127	100009999	1968,73	900029999	5503,08	2500049999	55076,2	10000099999	530188	90000299999	1431660	2,5E+11
Heap Sort	2,7592	518705	8,7699	1739633	19,4824	3056481	46,6272	6519813	127,902	21431637	175,502	37116275
Merge Sort	1,8889	1431110	6,5497	6691262	11,0356	16720383	24,2663	37104342	90,0728	99359047	137,091	217412214
Quick Sort	0,4261	254764	3,779	855622	1,8849	1508351	3,3012	3219401	11,198	10567016	19,6311	18332891
Radix Sort	0,3199	60107	1,1688	210131	2,2355	350131	3,2913	700131	13,7003	2400155	22,3289	4000155
Counting Sort	0,1023	70005	0,3383	210005	1,3279	350005	1,3663	700005	2,9557	2100005	4,9697	3500005
Binary Insertion Sort	79,1308	50252260	632,637	450849494	1797,31	1251493960	14267,9	5003187890	148769	45010501462	406047	1,2502E+11
Shaker Sort	389,034	200000001	3311,6	1800000001	14107	5000000001	81458,4	20000000001	697884	1,8E+11	1963510	5E+11
Flash Sort	0,6865	103752	1,9107	311252	2,5955	518752	5,9086	1037502	25,004	3112502	32,9099	5187502

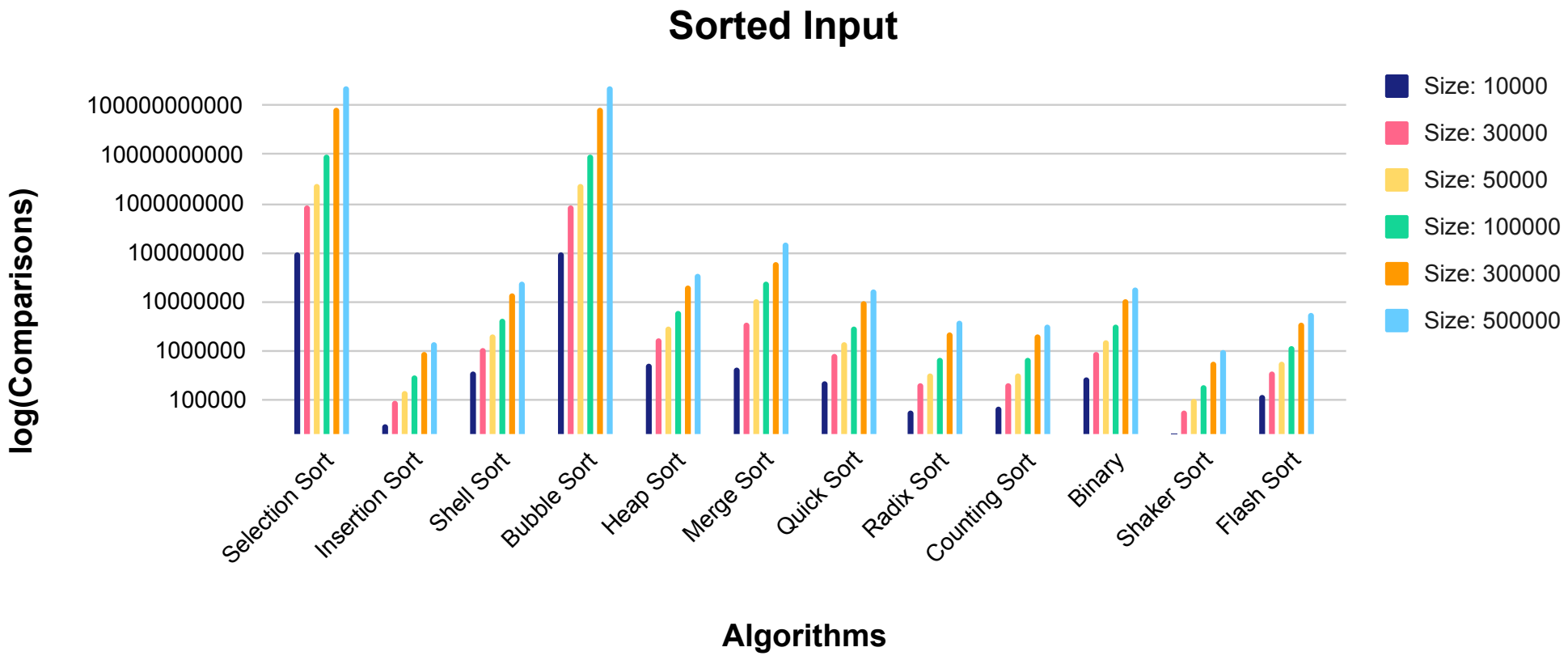
Data order: **Randomized**												
Data size	10.000		30.000		50.000		100.000		300.000		500.000	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision
Selection Sort	72,8876	100009999	675,09	900029999	1878,82	2500049999	12825,3	10000099999	220180	90000299999	728805	2,5E+11
Insertion Sort	130,055	49727376	1158,94	453956164	3102,37	1249929123	10960,5	4990211473	176560	45034384243	506017	1,251E+11
Shell Sort	2,7062	360042	9,2453	1170050	15,0898	2100049	32,5809	4500051	109,411	15300061	200,162	25500058
Bubble Sort	252,128	100009999	2342,79	900029999	11864,4	2500049999	47993,9	10000099999	666147	90000299999	2081340	2,5E+11
Heap Sort	5,033	517875	15,7985	1740722	28,8383	3059571	43,9528	6521229	142,695	21444835	260,311	36875028
Merge Sort	3,3413	2004825	14,3255	8599102	21,4229	20053445	43,5151	44170450	142,988	122441839	244,4	257294351
Quick Sort	0,9036	314867	4,7699	1126187	4,1647	2019129	9,22	4395007	28,7321	14774390	44,3427	28182172
Radix Sort	0,3412	60107	1,1721	210131	2,1878	350131	3,3941	700131	11,4715	2400155	20,378	4000155
Counting Sort	0,1238	69999	1,4104	210005	1,3746	315541	1,1992	565541	2,805	1565541	4,857	2565541
Binary Insertion Sort	33,8242	25162414	310,545	225998727	2143,52	622988577	10754,9	2506869309	96353,5	22539532279	329530	6,2562E+10
Shaker Sort	247,509	101180001	4425,13	895440001	11648,6	2499700001	56346,5	10061400001	688766	90054600001	2048100	2,4994E+11
Flash Sort	0,9527	96409	2,3163	288519	4,8677	466016	6,5136	881806	31,9216	2550156	70,7807	4543029

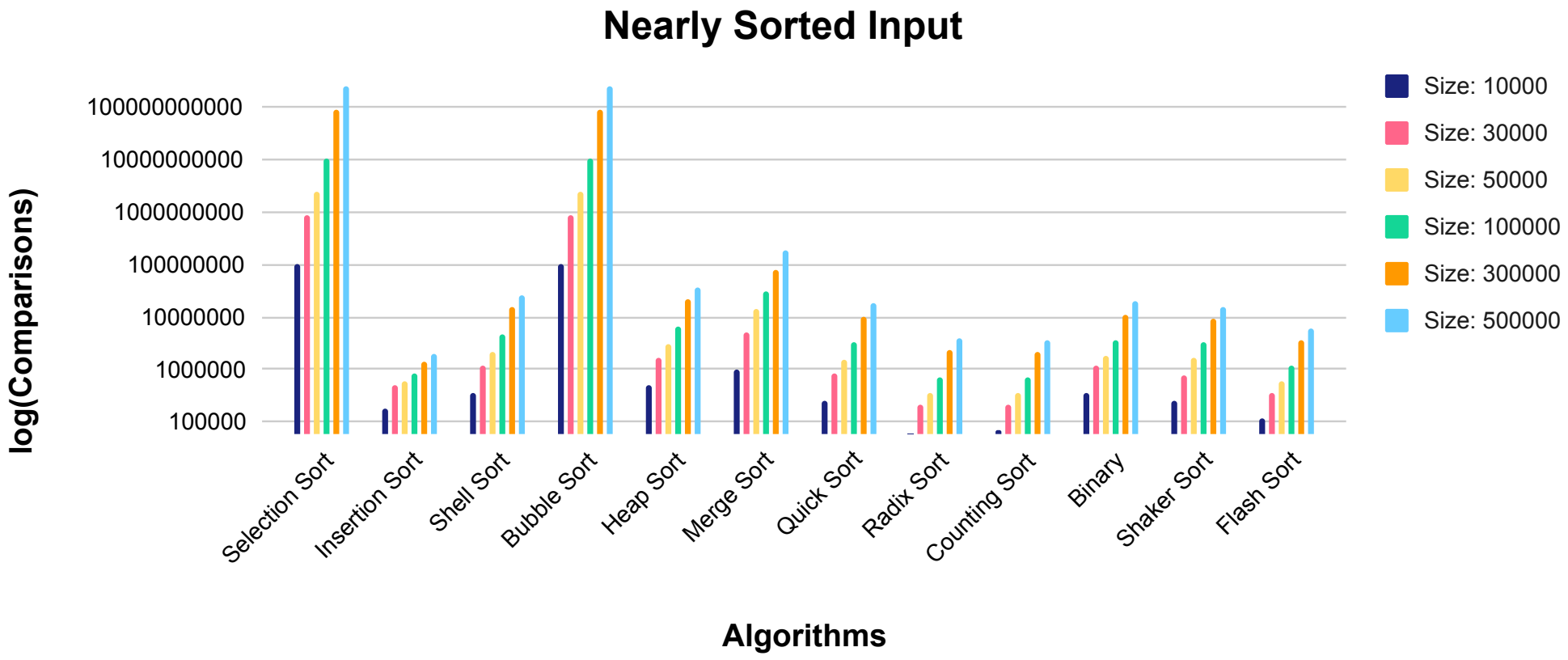

Sorted Input
Run time in log(seconds)
100
10
1
0.1
0.01
0.001
0.0001
10000
30000
50000
100000
300000
500000
Input Size
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary Insertion Sort
Shaker Sort
Flash Sort

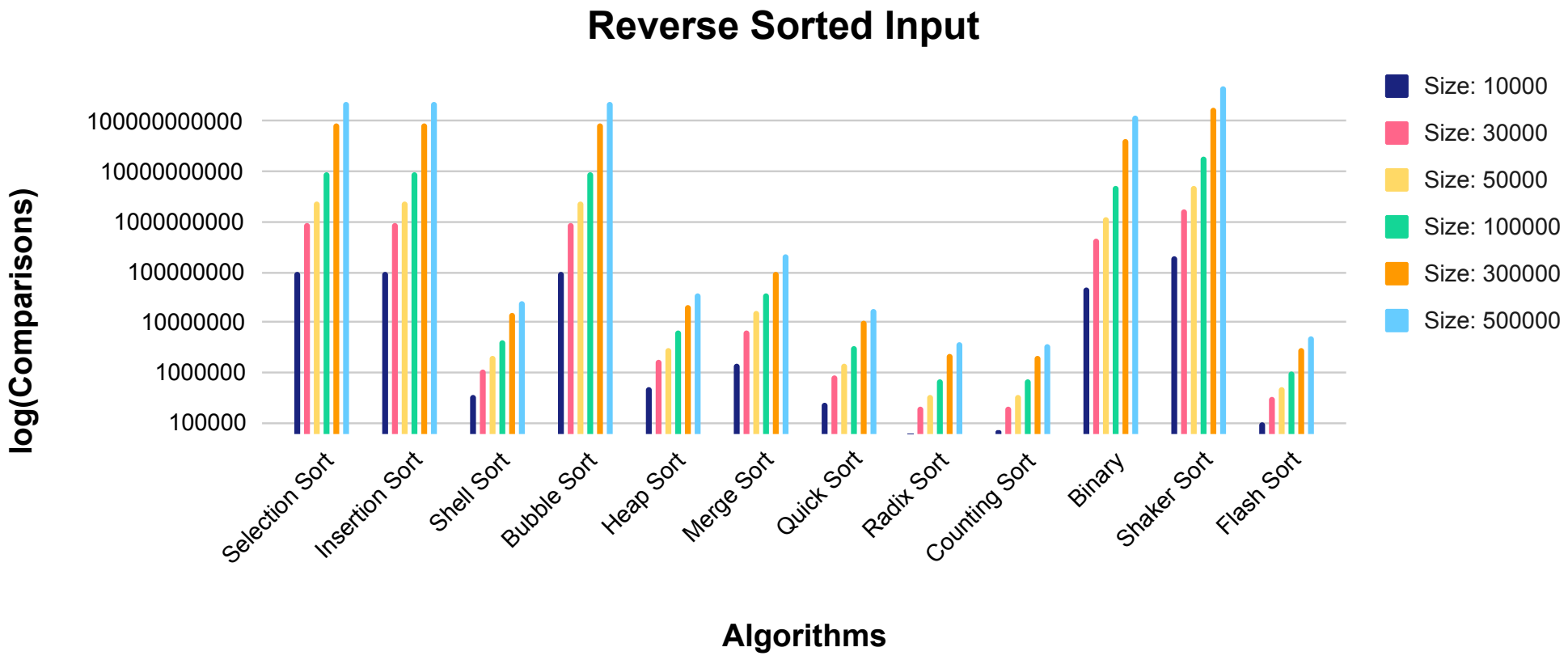

Nearly Sorted Input
Run time in log(seconds)
100
10
1
0.1
0.01
0.001
10000
30000
50000
100000
300000
500000
Input Size
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary Insertion Sort
Shaker Sort
Flash Sort

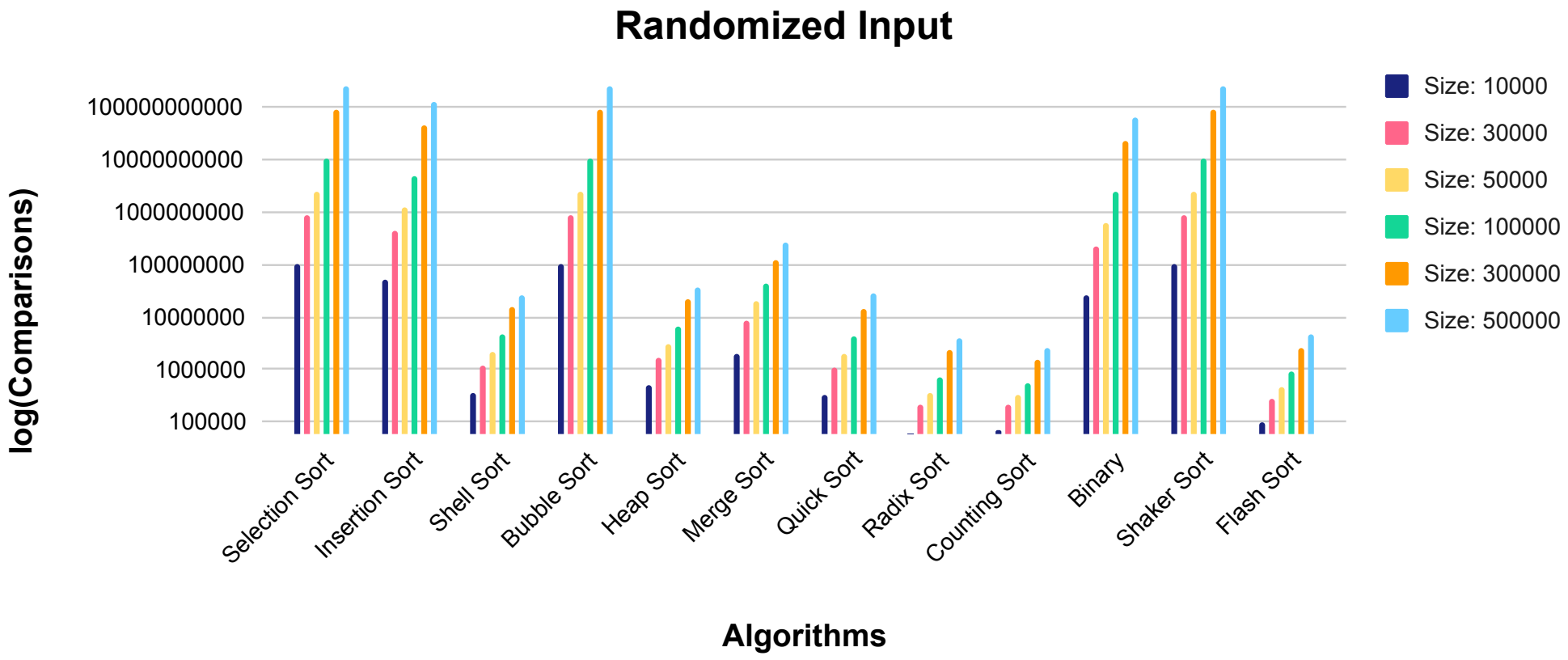

Reverse Sorted Input
Run time in log(seconds)
1000
100
10
1
0.1
0.01
0.001
10000
30000
50000
100000
300000
500000
Input Size
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary Insertion Sort
Shaker Sort
Flash Sort


Randomized Input
Run time in log(seconds)
1000
100
10
1
0.1
0.01
0.001
10000
30000
50000
100000
300000
500000
Input Size
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary Insertion Sort
Shaker Sort
Flash Sort


Sorted Input
log(Comparisons)
100000000000
10000000000
1000000000
100000000
10000000
1000000
100000
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary
Shaker Sort
Flash Sort
Algorithms
Size: 10000
Size: 30000
Size: 50000
Size: 100000
Size: 300000
Size: 500000


Nearly Sorted Input
log(Comparisons)
100000000000
10000000000
1000000000
100000000
10000000
1000000
100000
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary
Shaker Sort
Flash Sort
Algorithms
Size: 10000
Size: 30000
Size: 50000
Size: 100000
Size: 300000
Size: 500000


Reverse Sorted Input
log(Comparisons)
100000000000
10000000000
1000000000
100000000
10000000
1000000
100000
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary
Shaker Sort
Flash Sort
Algorithms
Size: 10000
Size: 30000
Size: 50000
Size: 100000
Size: 300000
Size: 500000


Randomized Input
log(Comparisons)
100000000000
10000000000
1000000000
100000000
10000000
1000000
100000
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary
Shaker Sort
Flash Sort
Algorithms
Size: 10000
Size: 30000
Size: 50000
Size: 100000
Size: 300000
Size: 500000

5 Acknowledgements

First and foremost, the authors express our sincere gratitude to Le Thanh Tung & Tran Hoang Quan for giving guidance.

In addition, we are deeply grateful to VNOI wiki, Google Scholar and Introduction to Algorithms (CLRS) book for providing us with the essential materials, papers, source codes and examples that served as the foundation for this project.

Large Language Models also help us understand a few concepts when we were stuck and give us some useful suggestions to improve certain aspects of our work.

Lastly, we appreciate all members of Group 07 for their effort, consistency, hard work, and collaborative spirit throughout this project.

6 References

References

[1] Rawan Ali Abdeen. An algorithm for string searching based on brute-force algorithm. *IJCSNS International Journal of Computer Science and Network Security*, 11(7):24–27, 2011.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition, 2022.

[3] Saqib Iqbal Hakak, Amirrudin Kamsin, Palaiahnakote Shivakumara, Gulshan Amin Gilkar, Wazir Zada Khan, and Muhammad Imran. Exact string matching algorithms: survey, issues, and future research directions. *IEEE access*, 7:69614–69637, 2019.